

# 影像處理期末報告： Tkinter 互動介面與 NumPy 幾何投影變換

林柏翰  
資工二甲  
國立澎湖科技大學

**Abstract**—本報告探討並實作了兩種基於 Python 的核心影像處理技術：其一為基於 Tkinter 與 PIL 的動態圖像旋轉縮放工具；其二為運用 NumPy 與線性代數實現的精確空間投影與圖層疊加工具。透過物件導向設計與單應性矩陣 (Homography Matrix) 的數學運算，本研究展示了如何在不依賴龐大外部引擎的情況下，達成商業等級的增強實境 (AR) 貼圖效果與高效能的圖形使用者介面狀態管理。

**Index Terms**—Image Processing, Tkinter, NumPy, Homography, Perspective Transform, Object-Oriented Programming

## I. INTRODUCTION

隨著電腦視覺技術的普及，影像的幾何變換與空間投影在介面設計中扮演著關鍵角色。本專案分為兩大模組：「圖像旋轉縮放工具」探討圖形使用者介面 (GUI) 的狀態管理機制與高效影像重採樣邏輯；「圖像疊加工具」則深入分析單應性矩陣的數學模型與多邊形遮罩合成技術。透過對比物件導向與函數式運算的優勢，本報告總結了跨平台影像處理的最佳實踐途徑。

## II. 圖像旋轉縮放系統設計

### A. 物件導向封裝與狀態驅動

本系統使用 ImageDecoratorApp 類別整合 UI 元件與處理邏輯，確保程式碼具備高內聚性與低耦合度。核心架構採用 current\_angle 與 current\_scale 變數實作輕量級狀態機。當使用者透過介面變更參數時，系統會立即偵測狀態變化並自動觸發畫布重繪，達成流暢的即時預覽效果。

### B. 重採樣與絕對置中演算

在影像變換的底層實作中，為減少邊緣鋸齒並保留細節，系統在縮放時採用 LANCZOS 演算法，旋轉時採用 BICUBIC 演算法。如圖 1 所示，為確保影像在任意角度下皆能完美定錨於畫布正中央，演算法透過精準計算邊界框 (Bounding Box) 改變後的位置偏移，並轉換為 RGBA 模式以確保擴展後的邊緣保持透明而非黑邊。

## III. 圖層疊加與透視投影實作

### A. 單應性矩陣投影與逆向映射原理

為了將商標完美貼合於傾斜的建築空地背景 (輸入素材如圖 2 所示)，本工具拋棄了傳統僅能處理平移、旋轉與等比例縮放的仿射變換 (Affine Transformation)，採用齊次座標系下的透視投影變換 (Perspective Transformation)，亦即單應性矩陣 (Homography Matrix) 映射技術。

單應性矩陣描述了一個平面到另一個平面的射影空間映射。在二維影像處理中，若直接採用正向映射 (Forward

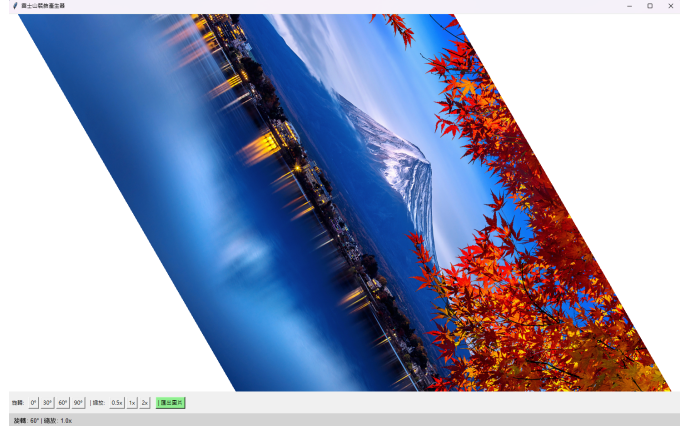


Fig. 1. 富士山圖像旋轉與縮放工具之執行結果。圖片經過  $-60^\circ$  旋轉與縮放後，依然完美定錨於畫布正中央並保留透明邊界。

Mapping)，常因離散座標取整而導致目標影像產生空洞與點位缺失。因此，本系統之核心設計採用逆向映射 (Inverse Mapping / Backward Warping) 機制，即計算從目標圖像座標  $(x_d, y_d)$  對應回來源商標座標  $(x_s, y_s)$  的變換關係。其齊次座標映射方程定義如下：

$$\begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix} \sim H^{-1} \begin{bmatrix} x_d \\ y_d \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \begin{bmatrix} x_d \\ y_d \\ 1 \end{bmatrix} \quad (1)$$

其中， $H^{-1}$  為透視變換矩陣。將右側矩陣乘法展開並消除非線性齊次因子後，可得到描述兩平面座標映射關係的非線性方程：

$$x_s = \frac{ax_d + by_d + c}{gx_d + hy_d + 1}, \quad y_s = \frac{dx_d + ey_d + f}{gx_d + hy_d + 1} \quad (2)$$

### B. 線性方程組建構與最小平方求解

為了求解矩陣中 8 個獨立的未知參數  $\mathbf{h} = [a, b, c, d, e, f, g, h]^T$ ，我們將上述非線性方程兩邊同乘以分母，並進行代數移項整理，將其轉化為對未知參數  $\mathbf{h}$  呈線性的代數方程組：

$$ax_d + by_d + c - gx_sx_d - hx_sy_d = x_s \quad (3)$$

$$dx_d + ey_d + f - gy_sx_d - hy_sy_d = y_s \quad (4)$$

由此可知，每一組對應點  $(s_i, d_i)$  皆能提供兩個獨立的線性約束方程。當給定四組不共線的控制頂點時，即可建構出一個固定解的線性方程組  $A\mathbf{h} = \mathbf{b}$ ，其中係數矩陣  $A$

的維度為  $8 \times 8$ ，觀測向量  $\mathbf{b}$  的維度為  $8 \times 1$ 。其具體矩陣結構如下：

$$\begin{bmatrix} x_{d1} & y_{d1} & 1 & 0 & 0 & 0 & -x_{s1}x_{d1} & -x_{s1}y_{d1} \\ 0 & 0 & 0 & x_{d1} & y_{d1} & 1 & -y_{s1}x_{d1} & -y_{s1}y_{d1} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{d4} & y_{d4} & 1 & 0 & 0 & 0 & -x_{s4}x_{d4} & -x_{s4}y_{d4} \\ 0 & 0 & 0 & x_{d4} & y_{d4} & 1 & -y_{s4}x_{d4} & -y_{s4}y_{d4} \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \\ g \\ h \end{bmatrix} = \begin{bmatrix} x_{s1} \\ y_{s1} \\ \vdots \\ x_{s4} \\ y_{s4} \end{bmatrix} \quad (5)$$

在程式實作中，此矩陣建構邏輯完全對應於演算法內的 `matrix.append` 運算。最終，系統呼叫 NumPy 科學計算庫中的 `np.linalg.lstsq` 函式，利用奇異值分解 (Singular Value Decomposition, SVD) 執行最小平方方法 (Least Squares Method) 求解。此數學設計不僅能精準計算出透視變換係數，亦具備極高的數值穩定性。



Fig. 2. 圖層疊加系統之輸入素材 (左：目標背景建築空地，右：欲覆蓋之商標)

#### C. 精確邊界控制與遮罩融合

為解決直接透視貼圖容易產生不規則黑邊的技術痛點，本專案引入了精確的遮罩融合技術。系統首先建立與背景同寬高的 8-bit 灰階遮罩 (L 模式)，接著使用 `ImageDraw.polygon` 依據四角座標填入純白，精確定義有效貼合區域。圖 3 展現了經過透視轉換與多邊形遮罩合成後的最終完美貼圖成果。



Fig. 3. 精確邊界控制與透視貼圖之合成結果。透過遮罩技術進行裁切與融合，成功將商標平貼於立體透視視角下的空地。

#### IV. 技術對比與總結

本專案成功展示了如何結合 Python 的科學計算能力與創意影像繪圖完成高複雜度運算。兩種工具的架構對比如表 I 所示：

TABLE I  
核心技術與架構對比

| 專案項目  | 圖像旋轉縮放工具              | 圖層疊加工具             |
|-------|-----------------------|--------------------|
| 技術焦點  | Tkinter UI / PIL 基礎變換 | NumPy 矩陣 / 單應性投影   |
| 數學複雜度 | 低 (線性仿射)              | 高 (投影幾何方程)         |
| 架構特色  | OOP 類別封裝、狀態機設計        | Functional 流程、資料驅動 |

#### V. CONCLUSION AND FUTURE WORK

本報告詳述了影像處理的核心技術與實作細節，證實了高度模組化且無需龐大外部引擎的輕量化架構，亦能實現商業級視覺成果。未來的優化方向將朝向提升互動性與自動化邁進：首先，計畫導入即時拖拽 (Drag & Drop) 功能，允許使用者透過滑鼠動態調整四個透視點；其次，將評估加入 OpenCV 進行特徵點自動辨識，以期實現無需手動輸入座標的自動化圖層合成。